

Unit 5.Queue

5.1 Concepts of Queue(FIFO)

5.1.1 Concepts of Queues and its basic operations.

5.2 Implementation of Queue

5.2.1 Simple Queue: insert, delete and display.

5.2.2 Double ended Queue: insert, delete and display.

5.2.3 Circular queue: Insert, delete and display.

5.1 Concepts of Queue(FIFO)

5.1.1 Concepts of Queues and its basic operations

A queue is a linear structure that follows a specific order in which operations are performed. The order is First In, First Out (FIFO). A prime example of a queue is any line of consumers waiting for a resource, where the consumer who arrived first is served first. The distinction between stacks and queues lies in the removal process. In a stack, we remove the item that was most recently added; in a queue, we remove the item that was least recently added.



5.2 Implementation of Queue

5.2.1 Simple Queue: insert, delete and display

OPERATIONS ON A QUEUE

The basic operations that can be performed on queue are :

1. To Insert an element in a Queue
2. To Delete an element from a Queue.
3. To Display(Traverse) all elements of a Queue.

An array $Q[\text{max}]$ contains a maximum of MAX elements. The indices F and R indicate the positions of the front and rear elements, respectively. The variable Num represents the number of elements currently inserted in the queue. Initially, both F and R are set to 0.

1) Insert an element in a queue

Step 1 :[Check for Queue Overflow]

If $R \geq \text{MAX}$ then

Write "Queue Overflow"

return

Step 2 :[Increment rear pointer]

$R = R + 1$

Step 3 :[Insert element]

Q[R] = NUM

Step 4 :[Set front pointer]

If F == 0 then

F=1

return

2) Delete an element from a queue

Step 1 : [Check for Queue Underflow]

If F == 0 then

Write "Queue Underflow"

return

Step 2 : [Display Deleted element]

Write "Deleted" Q[F]

Step 3 : [Set front pointer]

If F==R then

F = R = 0

Else

F = F + 1

Example 5.1 Simple Queue implementation

```
#include<iostream.h>
```

```
#include<conio.h>
```

```
#include<stdlib.h>
```

```
class queue
```

```
{
```

```
int q[10];
```

```
int r,f,max;
```

```
public:
```

```
queue()
```

```
{
```

```
cout<<"\n enter max";
```

```
cin>>max;
```

```
r = 0;
```

```
f = 0;
```

```
}
```

```
void insert(int x)
```

```
{
```

```
if(r >= max)
```

```
{
```

```
cout << "queue over flow";
```

```
return;
```

```
}
```

```
r++;
```

```

        q[r] = x;
        if(f==0)
            f=1;
    }

    void delet()
    {
        if(f==0)
        {
            cout << "queue underflow";
            return;
        }
        cout << "deleted " << q[f];

        if(f==r)
            f=r=0;
        else
            f++;
    }
    void display();
};

void queue::display()
{
    int i;
    if(f==0)
    {
        cout << " queue empty";
        return;
    }
    for(i = f; i<=r; i++)
        cout << q[i] << " ";
}

void main()
{
    clrscr();
    int ch,e;
    queue qe;
    while(1)
    {
        clrscr();
        cout << "\n 1.insert";
        cout << "\n 2.delet";
        cout << "\n 3.display";
        cout << "\n 4.exit";
        cout << "\n enter ur choice: ";
        cin >> ch;
        switch (ch)

```

```

{
case 1:
cout << "enter the element: ";
cin >> e;
qe.insert(e);
break;
case 2:
qe.delet();
break;
case 3:
qe.display();
break;
case 4:
exit(0);
default:
cout<<"\nwrong choice";
}
getch();
}
}

```

5.2.3 Circular queue: Insert, delete and display.

In a simple queue, memory usage can be inefficient in some situations. For instance, if we add 3 elements to a queue that has a capacity of 3, the queue becomes full. If we then remove 1 element, we cannot add new elements until the queue is completely empty, even though there is 1 available space. In contrast, a circular queue allows us to use the space left by deleted elements to insert new ones.

An array $Q[N]$ contains a maximum of N elements. The indices F and R indicate the positions of the front and rear elements, respectively. The variable Num represents the number of elements currently inserted in the queue. Initially, both F and R are set to 0.

1) Insert an element in a queue

Step1: [Check for Queue Overflow]

```

If (F==1 and R==N) OR (F==R+1) then
    Write "queue overflow "
    return

```

Step2 :[Reset rear pointer]

```

If R==N then
    R=1
Else
    R=R+1

```

Step 3: [Insert element]

```

Q[R]=NUM

```

Step 4: [Set front pointer]

 If F==0 then

 F=1

2) Delete an element from a queue

Step 1:[Check for Underflow]

 If F==0 then

 Write “queue underflow”

 return

Step 2 :[Display Deleted element]

 Write “Deleted ”,Q[F]

Step 3:[Reposition F and R]

 If F==R then

 F=R=0

 Else if F==N then

 F=1

 Else

 F=F+1

Step 4 : Exit

Example 5.2 circular queue

```
#include<iostream.h>
```

```
#include<conio.h>
```

```
#include<stdlib.h>
```

```
class cir
```

```
{
```

```
  int Q[10];
```

```
  int r, f,n;
```

```
  public:
```

```
    cir()
```

```
    {
```

```
        cout<<"enter value of n";
```

```
        cin>>n;
```

```
        r = 0;
```

```
        f = 0;
```

```
    }
```

```
  void insert(int num)
```

```
  {
```

```
    if((f==1 && r==n)||((f==r+1))
```

```
    {
```

```

        cout<<"\n overflow";
        return;
    }

    if(r==n)
        r=1;
    else
        r++;

    Q[r]=num;
    if(f==0)
        f=1;
}

void del()
{
    if(f==0)
    {
        cout<<"\n underflow ";
        return;
    }
    cout<<"\n Element deleted:"<<Q[f];
    if(f==r)
    {
        f=0;
        r=0;
    }
    else if(f==n)
        f=1;
    else
        f++;
}

void display();

void cir::display()
{
    int i;
    if(f==0)
    {
        cout<<"\n queue is empty ";
        return;
    }
}

```

```

    }
    else if(r>=f)
    {
        for(i=f;i<=r;i++)
            cout<<Q[i]<<" ";
    }
    else
    {
        for(i=1;i<=r;i++)
            cout<<Q[i]<<" ";
        for(i=f;i<=n;i++)
            cout<<Q[i]<<" ";
    }
}

void main()
{
    int ch,e;
    clrscr();
    cir qu;
    while(1)
    {
        clrscr();
        cout << "\n1.insert";
        cout << "\n2.delet";
        cout << "\n3.display";
        cout << "\n4.exit";
        cout << "\nenter ur choice: ";
        cin >> ch;
        switch (ch)
        {
            case 1:
                cout<<"\n Enter element :";
                cin>>e;
                qu.insert(e);
                break;
            case 2:
                qu.del();
                break;
            case 3:
                qu.display();
                break;
            case 4:

```

```

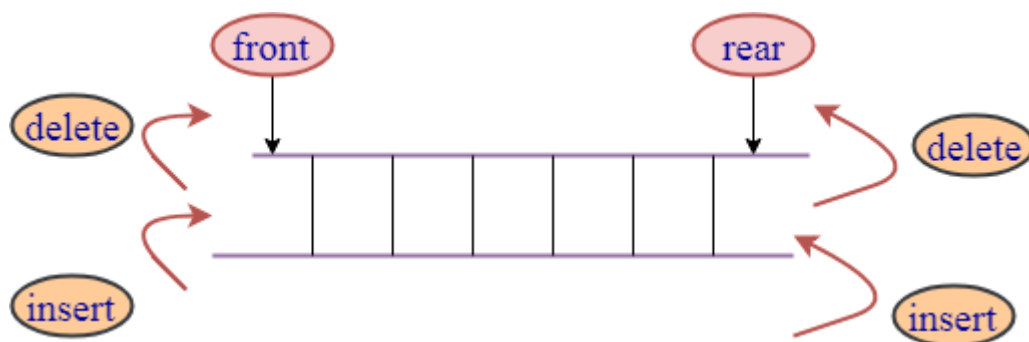
        exit(0);
        default:
            cout<<"\n wrong choice";
    }
    getch();
}
}

```

5.2.2 Double ended Queue: Insert, delete and display

Deque

Double ended queue or simply called “Deque”. The difference between Queue and Deque is that it does not follow the FIFO (First In, First Out) approach. The second feature of Deque is that we can insert and remove elements from either front or rear ends.



OPERATIONS ON A CIRCULAR DEQUE

The basic operations that can be performed are :

1. To Insert an element at the front in a Queue.
2. To Insert an element at the rear in a Queue.
3. To Delete an element from the front of Queue.
4. To Delete an element from the rear of Queue.

An array $Q[N]$ contains a maximum of N elements. The indices F and R indicate the positions of the front and rear elements, respectively. The variable $item$ represents the number of elements currently inserted in the queue. Initially, both F and R are set to -1 .

- 1) To Insert an element at the front in a Queue.

Step 1 : [Check for queue overflow]

If $((f == 0) \ \&\& \ (r == N - 1)) \ || \ (f == r + 1)$ then

Write “Queue Overflow”

return

[set f and r pointer]

Else if (f==-1) then

f=N-1

r=N-1

Else if (f==0) then

f=N-1

Else

f=f -1

Step 2 : Set Q[f] = item

Step 3 : Exit

2) To Insert an element at the rear in a Queue.

Step 1 :[Check for queue overflow]

If (((f==0) && (r==N-1)) || (f==r+1)) then

Write "Queue Overflow"

return

[set f and r]

Else if (f==-1) then

f=0

r=0

Else if (r==N-1) then

r=0

Else

r=r +1

Step 2 : Set Q[r] = item

Step 3 : Exit

3) To Delete an element from the front of Queue

Step 1 : [Check for queue underflow]

If f == -1 then

Write "Queue Underflow"

return

Step 2 : [Assign Deleted element]

item = Q[f]

Step 3 : [Reset f and r]

If (f ==r) then

f= r= -1

Else if (f== N-1)

f=0

Else

f=f+1

Step 4 : [Return element]

return item

Step 5: Exit

4) To Delete an element from the rear of Queue.

Step 1 : [Check for queue underflow]

 If $f == -1$ then

 Write "Queue Underflow"

 return

Step 2 : [Assign Deleted element]

 item = Q[r]

Step 3 : [Reset f and r]

 If $(f == r)$ then

$f = r = -1$

 Else if $(r == 0)$

$r = N - 1$

 Else

$r = r - 1$

Step 4 : [Return element]

 return item

Step 5: Exit

Example 5.3 circular dequeue

```
#include<iostream.h>
```

```
#include<stdlib.h>
```

```
#include<conio.h>
```

```
#define N 3
```

```
class deq
```

```
{
```

```
    private:
```

```
        int Q[N];
```

```
        int f,r;
```

```
    public:
```

```
        deq()
```

```
        {
```

```
             $f = -1$ ;
```

```
             $r = -1$ ;
```

```
        }
```

```
        void insert_frontend(int item);
```

```
        void insert_rearend(int item);
```

```
        int delete_frontend();
```

```
        int delete_rearend();
```

```
        void display();
```

```
};
```

```
void deq::insert_frontend(int item)
```

```
{
```

```
    if ((( $f == 0$ ) && ( $r == N - 1$ )) || ( $f == r + 1$ ))
```

```
    {
```

```
        cout<<"\nqueue overflow\n";
```

```
        return;
```

```
    }
```

```

        else if( f==-1 )/*if queue is initially empty*/
        {
            f=N-1;
            r=N-1;
        }
        else if(f==0) /* front is at first position of queue*/
            f=N-1;
        else
            f=f-1;
            Q[f]=item ;
    }/*end of insert_frontend()*/

void deq::insert_rearend(int item)
{
    if(((f==0) && (r==N-1)) || (f==r+1) )
    {
        cout<<"\nqueue overflow\n";
        return;
    }
    else if(f==-1) /*if queue is initially empty*/
    {
        f=0;
        r=0;
    }
    else if(r==N-1) /*rear is at last position of queue */
        r=0;
    else
        r=r+1;
    Q[r]=item ;
}/*end of insert_rearend()*/

int deq::delete_frontend()
{
    int item;
    if(f == -1)
    {
        cout<<"\nqueue underflow\n";
        return(0);
    }
    item=Q[f];
    if(f==r) /*queue has only one element */
    {
        f=-1;
        r=-1;
    }
    else if(f==N-1) /* front is at last position of queue*/
        f=0;
    else
        f=f+1;
}

```

```

        return item;
    } /*end of delete_frontend()*/
int deq::delete_rearend()
{
    int item;
    if(f==-1)
    {
        cout<<"\nqueue underflow\n";
        return(0);
    }
    item=Q[r];
    if(f==r) /*queue has only one element*/
    {
        f=-1;
        r=-1;
    }
    else if(r==0) /* rear is at first poition of queue*/
        r=N-1;
    else
        r=r-1;
    return item;
} /*end of delete_rearend() */

void deq::display()
{
    int i;
    if( f == -1)
    {
        cout<<"\nqueue underflow\n";
        return;
    }
    cout<<"\nqueue elements :\n";
    if(f<=r)
    {
        for(i=f;i<=r;i++)
            cout<<" "<<Q[i];
    }
    else
    {
        for(i=0;i<=r;i++)
            cout<<" "<<Q[i];
        for(i=f;i<=N-1;i++)
            cout<<" "<<Q[i];
    }
    cout<<"\n";
} /*end of display() */
/* c program to implement deque using circular array */
void main()
{

```

```

    deq q1;
    int choice,item;
    do
    {
        clrscr();
        cout<<"\n\n1.insert at the front end\n";
        cout<<"2.insert at the rear end\n";
        cout<<"3.delete from front end\n";
        cout<<"4.delete from rear end\n";
        cout<<"5.display\n";
        cout<<"6.Exit\n";
        cout<<"\nenter your choice : ";
        cin>>choice;
        switch(choice)
        {
            case 1:
                cout<<"\ninput the element for adding in queue : ";
                cin>>item;
                q1.insert_frontend(item);
                break;
            case 2:
                cout<<"\ninput the element for adding in queue : ";
                cin>>item;
                q1.insert_rearend(item);
                break;
            case 3:
                cout<<"\nelement deleted from front end is :\n"<<q1.delete_frontend();
                break;
            case 4:
                cout<<"\nelement deleted from rear end is : "<<q1.delete_rearend();
                break;
            case 5:
                q1.display();
                break;
            case 6:
                cout<<"\nHave a nice day\n";
                break;
            default:
                cout<<"\nwrong choice\n";
                break;
        }/*end of switch*/
        getch();
    }while(choice!=6);/*end of while*/
}/*end of main()*/

```

Deque can be classified as follows:

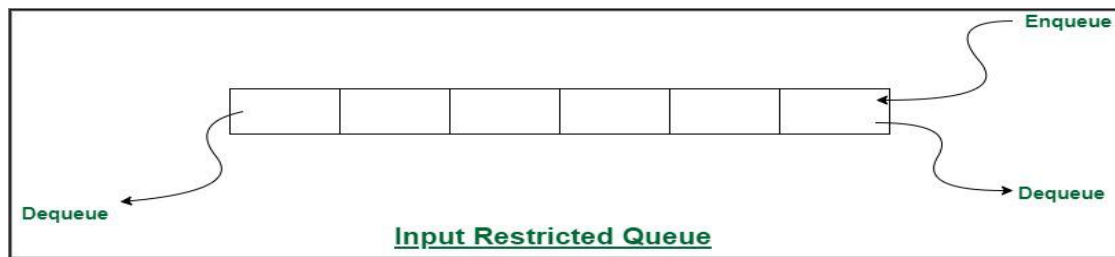
Input-restricted Deque: In input-restricted, deletion can be done from both the ends but insertion can be done only at one end of the queue.

The following three basic operations are performed on input restricted queue:

insertRear(): Adds an item at the rear of the queue.

deleteFront(): Deletes an item from the front of the queue.

deleteRear(): Deletes an item from the rear of the queue.



Output-restricted Deque: In the output-restricted queue, insertion can be done from both the ends but deletion is done only at one end i.e. the front end of the queue.

The following three basic operations are performed on output restricted queue:

insertRear(): Adds an item at the rear of the queue.

insertFront(): Adds an item at the front of the queue.

deleteFront(): Deletes an item from the front of the queue.

